

L2 ▪ Slack 接入与消息管道

- 对应代码：session-02-slack-api/ (verify_signature.py、echo_server.py、echo_server_ws.py)
- session-03-pipeline-agent/ (pipeline.py、slack_pipeline_demo.py、slack_adapter.py)
- common/message_pipeline.py、common/message_store.py

已邀请加入slack的名单（检查邮件确认）：

Liohorz233, liohorz233@gmail.com

[Mikaelaliang999](#), mikaelaliang999@gmail.com

william.jj.liu@gmail.com, william.jj.liu@gmail.com

课后学员能够

- 1. 理解 HMAC 签名，说清为什么校验必须包含时间戳
- 2. 对比 HTTP Events API 与 Socket Mode，能根据本地开发 / 生产部署场景选择接入方式
- 3. 理解「3 秒 ack + 异步处理」的时序图，并说明不这么做会发生什么
- 4. 把一坨 if-else 的消息处理重构成中间件管道，并给新中间件写单测
- 5. 跑通「普通消息 → Pipeline → 存储 → 摘要命令 → thread 回帖」，说明 mock 与真实接入分别验证了什么

Slack/App/Bot 权限、事件订阅、代码监听

基本概念：

Slack Service: Slack IM 服务，目前已经变成了一个流行的企业团队合作平台，类似企业微信

Slack App: Slack的扩展方式，类似wechat小程序

Bot: slack workflow自动化在slack会话中的代理用户

Channel/thread: Bot与人交互的信息传递媒介

权限：

- 有权限表示允许访问，不代表 Slack 已经推送，也不代表程序已经处理
- Bot OAuth 权限：已确认具有 `channels:history`、`app_mentions:read`、`chat:write`
- App 后台 Event Subscriptions: 需订阅 `message.channels` 与 `app_mention`，每期开课都要核对，不能只看 `token scopes`
- Python 事件监听：新 `slack_adapter.py` 已同时监听 `message` 和 `app_mention`；普通消息保存，提及走命令分支
- Bot 是否已加入目标频道：新频道仍需邀请 Bot，不能理解为可读取所有公开频道

已授予的 Bot Scope 与课程的关系

Scope	能力	与课程的关系
app_mentions:read	读取直接提及 App 的消息	@bot hello 演示；后续 @bot summary 触发摘要
chat:write	通过 Web API 发送消息	L2 回 echo；L3 集成后回传摘要
channels:history	读取已加入公开频道消息	支持 message.channels 事件；L2/L3 收集与摘要的权限基础
channels:read	查看公开频道基本信息	选择演示频道、频道 ID 对应名称
users:read	查询用户资料	把用户 ID 转成显示姓名，摘要更易读
channels:manage	管理公开频道	当前链路不需要，已授予不代表必须使用

关键区分

Bot 配置

Basic Information (Signing Secret 在这页)

: <https://api.slack.com/apps/A0BF4E57QE9/general>

OAuth & Permissions (Bot Token / scope 列表)

: <https://api.slack.com/apps/A0BF4E57QE9/oauth>

Socket Mode (开关 + App-Level Token / connections:write)

: <https://api.slack.com/apps/A0BF4E57QE9/socket-mode>

Event Subscriptions (订阅 message.channels 与 app_mention)

: <https://api.slack.com/apps/A0BF4E57QE9/event-subscriptions>

Install App (重新安装以生效新 scope)

: <https://api.slack.com/apps/A0BF4E57QE9/install-on-team>

已实现的交互

- 用户在指定公开测试频道正常讨论 → Bot 接收普通消息
- L2: 过滤、去重、清洗、保存统一消息记录
- 用户发送 @slack_assistant summary
- L3: 读取指定频道已保存的记录, 生成摘要
- → chat.postMessage 将摘要回复到触发消息的 thread
- 命令和数据分开: 摘要命令、Bot 自己的回复不进入摘要材料
- 新 demo 的摘要请求限流只作用于命令; 课堂练习的 rate_limit 作用于全部消息, 两者不要混用

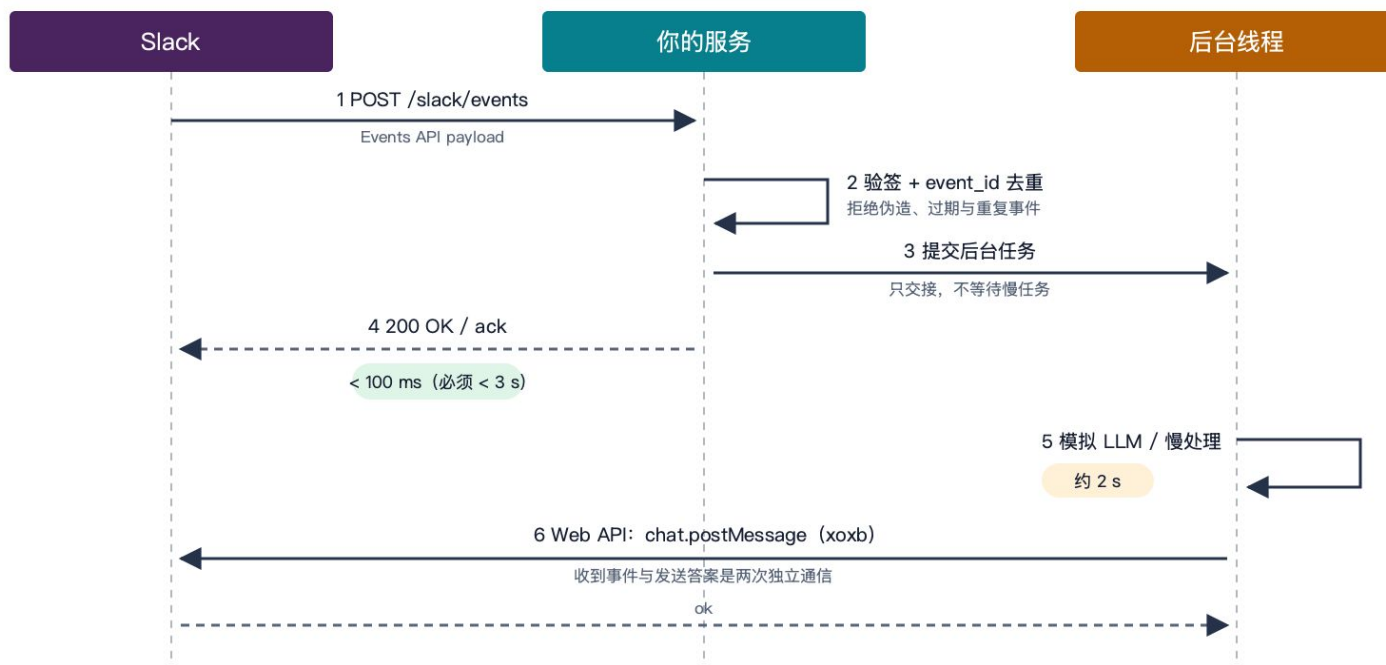
HTTP Events API vs Socket Mode

方式	谁发起连接	需要什么	适合场景
HTTP Events API	Slack POST 到你的服务	公网 HTTPS URL + SLACK_SIGNING_SECRET	托管生产、横向扩容、 Marketplace
Socket Mode	你的服务连 Slack WebSocket	SLACK_APP_TOKEN (connections:write)	本地开发、防火墙后、内部工具

HTTP Event

HTTP Events API: 先确认，再异步处理

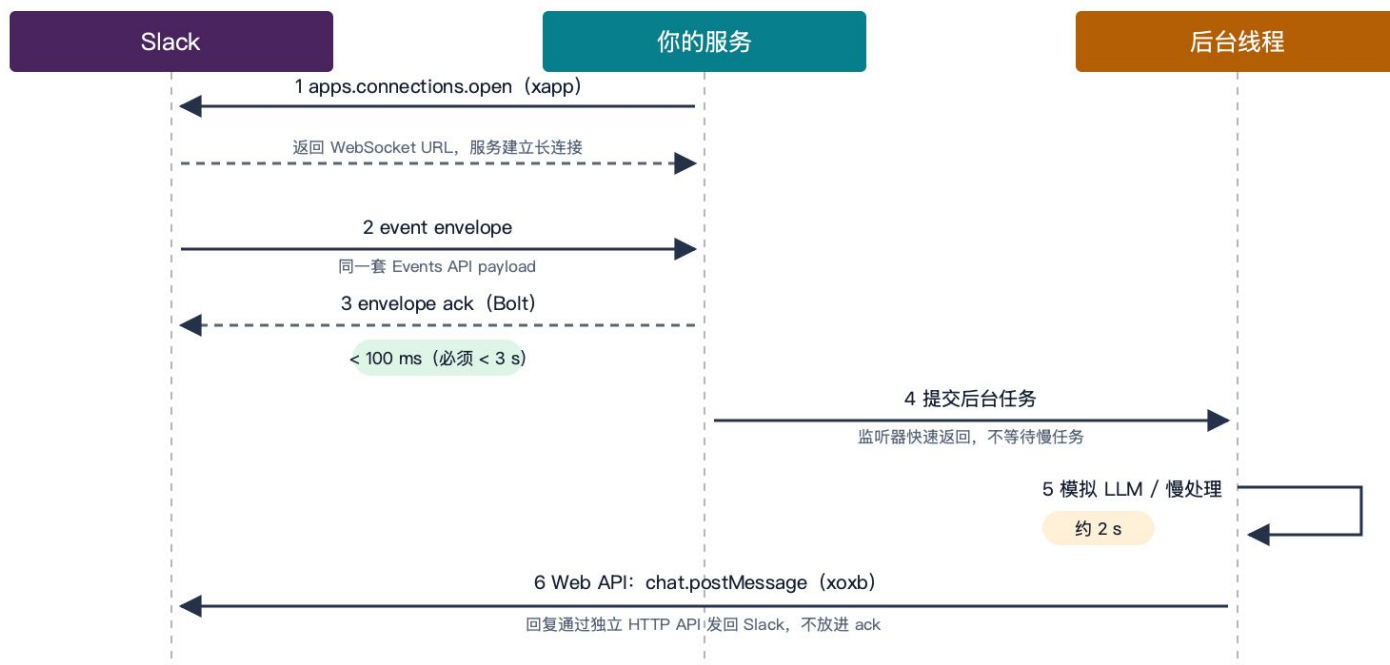
公网 Request URL · SLACK_SIGNING_SECRET 验签 · 三泳道时序



关键约束：ack 路径不等待后台处理；否则超过 3 秒，Slack 会判定失败并重发。

Socket Mode: WebSocket 入站, Web API 回帖

无需公网 URL · SLACK_APP_TOKEN 建连 · 三泳道时序



关键差异: 服务主动连接 Slack; 共同约束: 仍然先 ack, 再异步处理。

HTTP Event mode: 为什么签名校验不是可选项

“ 你的 bot 有一个公网 URL。我知道这个 URL。我能干什么？ ”

HTTP mode: HMAC 校验的三要素

- 签名基串 = "v0:" + timestamp + ":" + raw_body
- signature = "v0=" + HMAC_SHA256(signing_secret, 基串)
- 校验 = hmac.compare_digest(算出来的, 请求头里的) 且 $|\text{now} - \text{timestamp}| < 5\text{min}$
- 用 raw body, 不是解析后的 JSON —— 重新序列化可能改变字节, 导致签名不匹配
- 用 compare_digest, 不是 == —— 防时序攻击, ==不保证常数时间
- 时间戳窗口 —— 没有它攻击者可以重放一个合法的旧请求; 签名合法 $\neq$ 请求新鲜

3 秒 ack 是平台强加的约束

- 不管走 HTTP 还是 WebSocket, Slack 都要求快速确认收到
- HTTP 回 2xx; Socket Mode 回 envelope ack, Bolt 帮你处理
- 3 秒内不确认, Slack 就认为投递失败并重发
- LLM 调用五秒、十秒都正常 —— 必须先 ack, 再异步处理
- 收到事件和发送答案是两次独立通信: 先确认事件, 再用 `chat.postMessage` 主动发答案
- Slack 重发时带同一个 `event_id` —— 必须去重, 这是下半节课中间件管道要解决的问题之一

把 if-else 重构成中间件管道

代码: [common/message_pipeline.py](#)

```
def handle(msg):  
    if msg.get("bot_id"): return  
    if msg["id"] in seen: return  
    seen.add(msg["id"])  
    log(msg)  
    msg["text"] = msg["text"].strip().lower()  
    ... # more rules
```

- 坏代码的三个问题（顺序很重要，第三条最重要）：
 - 难以隔离测试 — 单测去重时混入其他规则和副作用
 - 不可插拔 — 加一条规则要改这个函数，改坏了影响全部规则
 - 规则顺序缺少解释 — 调换过滤/日志/去重可能改变行为，需要显式表达取舍
- 责任链模式：每个中间件拿到 next_，可以选择调用它（放行）或不调用（拦截）
- 顺序显式、可独立测试、加一层不用改别的层 —— Express / Django / gRPC 拦截器都是它

四个最小中间件

中间件	职责	为什么单独一层
ignore_bots	过滤 bot 自己的消息	不过滤会无限自我回复，是新人第一个生产事故
dedupe	基础示例按 ts 去重	新 demo 分开处理 event_id 和 (team_id, channel, ts)
logger	记录放行	出事时唯一能查的东西
normalize	清洗文本	下游所有模块都假设文本已清洗

主 demo：普通讨论到摘要回帖

- 先跑离线，观察链路：`slack_pipeline_demo.py --offline`
`python course-demos/session-03-pipeline-agent/slack_pipeline_demo.py --offline`
- 预期：6 条模拟素材保存、1 条合成摘要命令回复；查看 `messages.jsonl` / `trace.jsonl` / `summary_runs.jsonl`
- 切换真实传输：`--socket --channel C0C16PDBFHR --mock --seed short`（只由讲师执行一次）
- 学员发一条讨论 + `@slack_assistant summary`，观察原 `thread` 回复；普通消息只保存不回帖
- `posted=6` 只证明发送成功，需核对 `trace.jsonl` 中六条 `stored` 才证明接收成功
- 三处边界：来源白名单（不认 [模拟] 前缀）；`ack` 与业务分离；本节 `mock`，L3 换真实模型

课后练习：加一个 `rate_limit` 中间件

- 任务：在最小 `pipeline.py` 示例中加 `rate_limit`，滚动 60 秒窗口，同一作者最多 3 条，第 4 条拦截
- 范围区别：练习版本限制所有示例消息；实际 demo 的 `SummaryRateLimit` 只限制摘要请求 —— 不能混用
- 要求：注册位置自己决定并写明理由；加测试覆盖放行/拦截/另一作者/窗口恢复；本地测试通过后跑全量 `pytest`
- 讨论室重点：中间件顺序影响 `logger` 是否执行；测试只写 happy path；直接改 `handle()` 而非写成中间件

本节作业

- 作业 1: 完成 `rate_limit` 练习, 测试通过, 能解释中间件位置与两种限流的区别
- 作业 2: 画出 HTTP 与 Socket Mode 的时序图, 标出连接发起方、耗时量级、三种凭据位置
- 作业 3: 提交新 Pipeline demo 的链路证据 (`stored` / `replied` / `ignored` 各一条)
- 下节预告: 今天确认了消息如何变成摘要输入, 下节课检查摘要是否忠实、是否适合读者

把技术细节转成面试话术

- 如何设计应对第三方平台超时限制的异步架构? → `ack-then-process` + 幂等去重 + 回调发送
- 怎么保证 webhook 的安全性? → `HMAC` + `raw body` + 常数时间比较 + 时间戳窗口
- `HTTP Events API` 和 `Socket Mode` 怎么选? → 本地/防火墙后优先 `Socket`; 生产/横向扩容优先 `HTTP`
- 怎么证明消息链路跑通? → `posted` 与 `stored` 分开核对、`source` 区分真实和回放、快照追踪到摘要输入
- 高流量/错误处理 - `what if no ack`, `what if bot crash` ...